
Airflow Tasks

Expected Workflow

wake_up_task

↓

attend_class_task

↓

complete_assignment_task

↓

sleep_task

After setting up killercoda and opening Airflow UI,

In killercoda Terminal, run

docker exec -it airflow bash

cd /opt/airflow/dags

To open student_daily_tasks_dag.py file, run:

cat > student_daily_tasks_dag.py

```
from datetime import datetime
```

```
from airflow import DAG
```

```
from airflow.operators.python import PythonOperator
```

```
def wake_up():
```

```
    print("Student woke up at 6 AM")
```

```
def attend_class():
```

```
    print("Student attended Python class")
```

```
def complete_assignment():
```

```
    print("Student completed Airflow assignment")
```

```
def sleep():
```

```
    print("Student went to sleep at 10 PM")
```

```
default_args = {
```

```
    'start_date': datetime(2026, 1, 1),
```

```
}
```

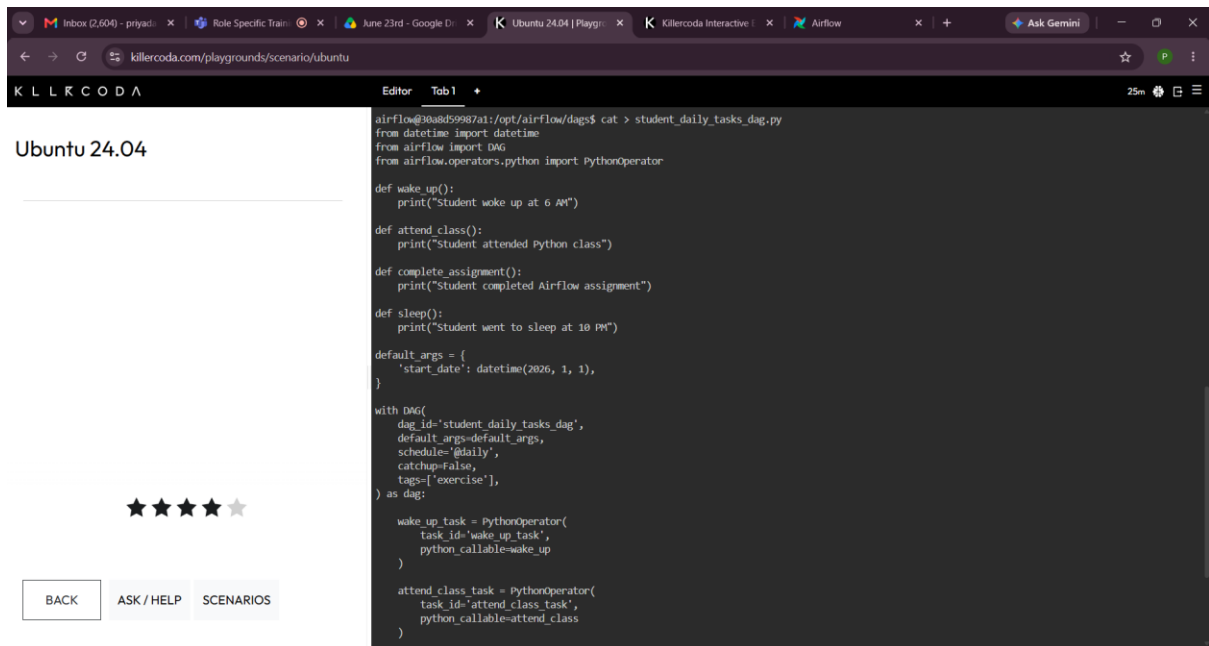
```

with DAG(
    dag_id='student_daily_tasks_dag',
    default_args=default_args,
    schedule='@daily',
    catchup=False,
    tags=['exercise'],
) as dag:
    wake_up_task = PythonOperator(
        task_id='wake_up_task',
        python_callable=wake_up
    )
    attend_class_task = PythonOperator(
        task_id='attend_class_task',
        python_callable=attend_class
    )
    complete_assignment_task = PythonOperator(
        task_id='complete_assignment_task',
        python_callable=complete_assignment
    )
    sleep_task = PythonOperator(
        task_id='sleep_task',
        python_callable=sleep
    )

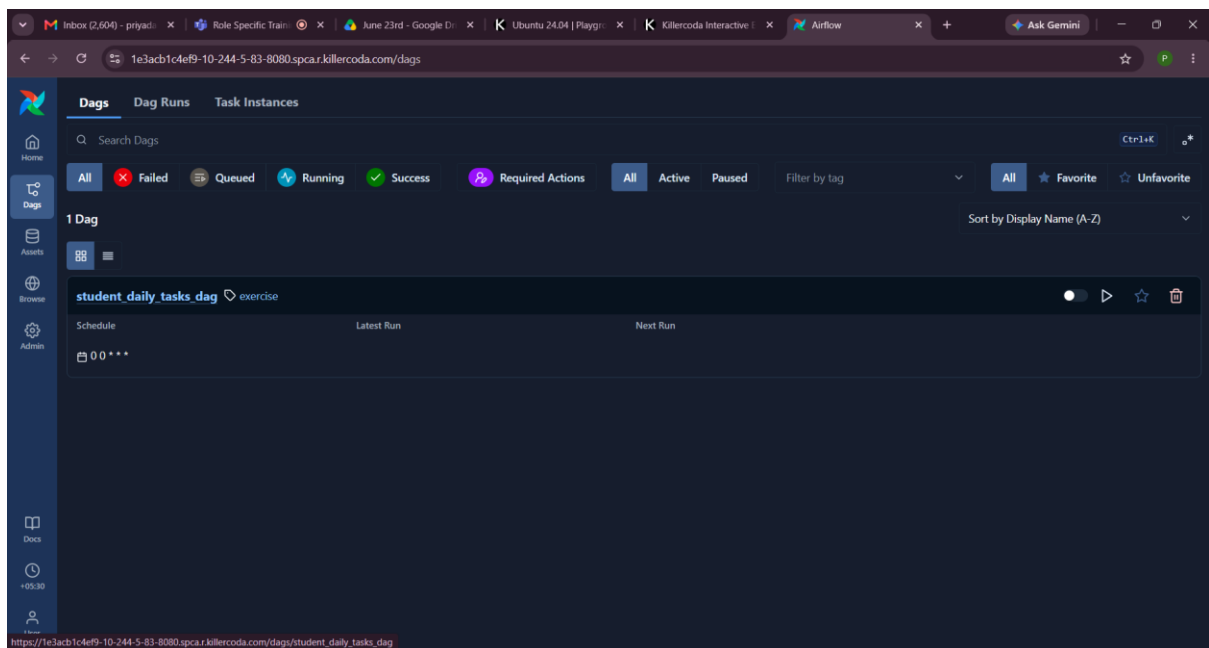
    wake_up_task >> attend_class_task >> complete_assignment_task >> sleep_task

```

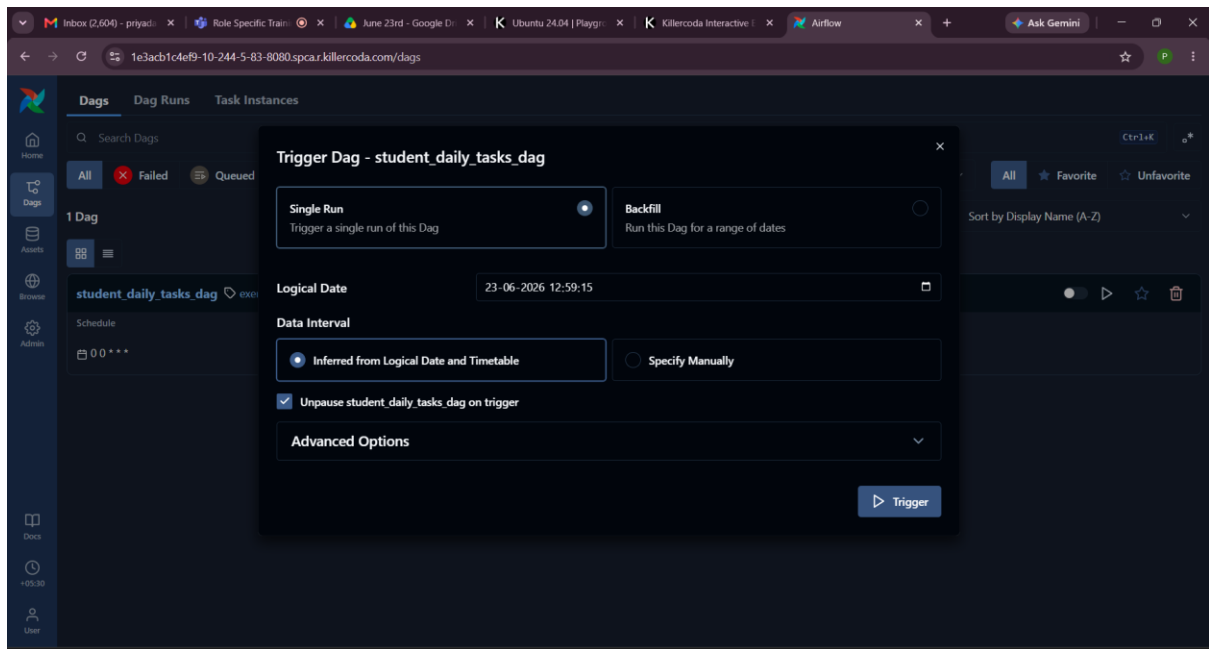
After completing the code, press **ctrl + d** to save



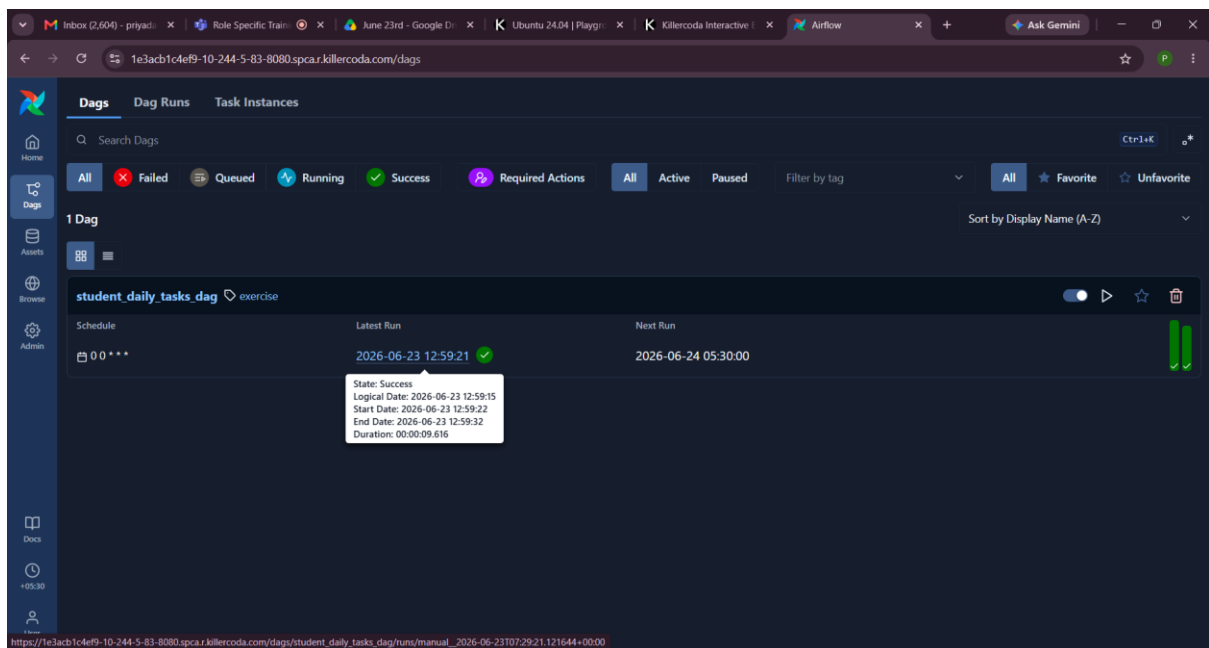
Then, refresh the Airflow UI and navigate to dags,



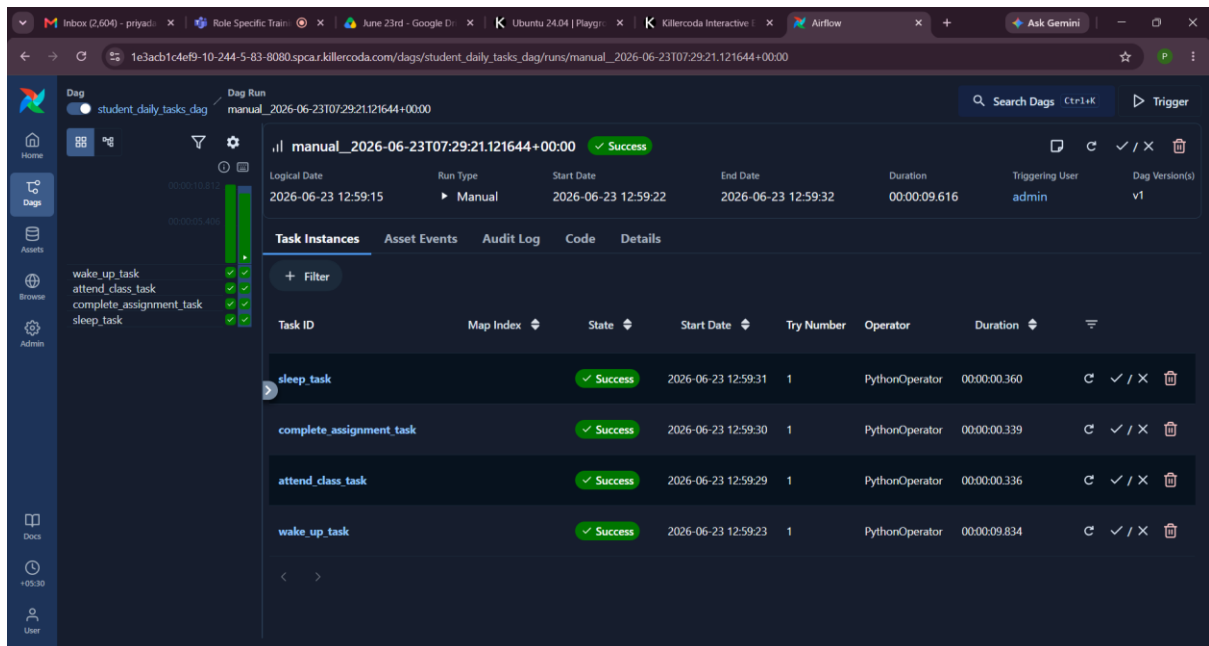
Press the run button, and click on the trigger to execute student_daily_tasks_dag



After completing the execution,



Click on the latest run to verify the execution of tasks,



Additional Task,

Add a new task:

exercise_task

with output:

Student completed morning exercise

Place it between:

wake_up_task and attend_class_task

Make changes in the code of student_daily_tasks_dag.py to add this additional task,

from datetime import datetime

from airflow import DAG

from airflow.operators.python import PythonOperator

def wake_up():

 print("Student woke up at 6 AM")

def exercise():

 print("Student completed morning exercise")

def attend_class():

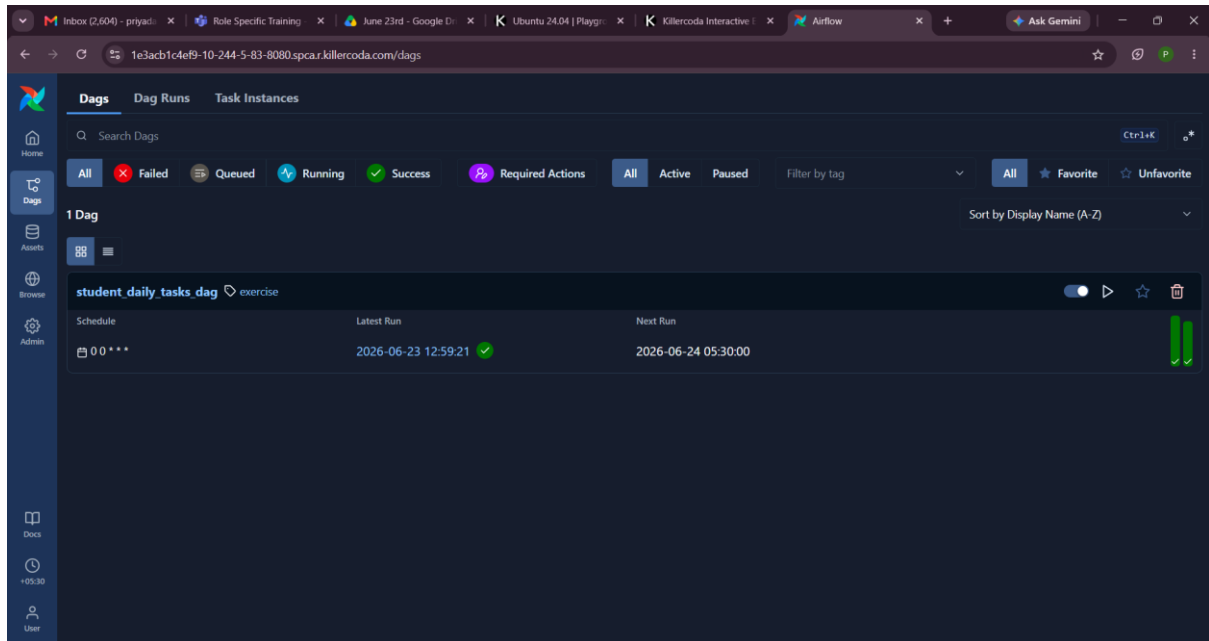
```

    print("Student attended Python class")
def complete_assignment():
    print("Student completed Airflow assignment")
def sleep():
    print("Student went to sleep at 10 PM")
default_args = {
    'start_date': datetime(2026, 1, 1),
}
with DAG(
    dag_id='student_daily_tasks_dag',
    default_args=default_args,
    schedule='@daily',
    catchup=False,
    tags=['exercise'],
) as dag:
    wake_up_task = PythonOperator(
        task_id='wake_up_task',
        python_callable=wake_up
    )
    exercise_task = PythonOperator(
        task_id='exercise_task',
        python_callable=exercise
    )
    attend_class_task = PythonOperator(
        task_id='attend_class_task',
        python_callable=attend_class
    )
    complete_assignment_task = PythonOperator(
        task_id='complete_assignment_task',
        python_callable=complete_assignment
    )

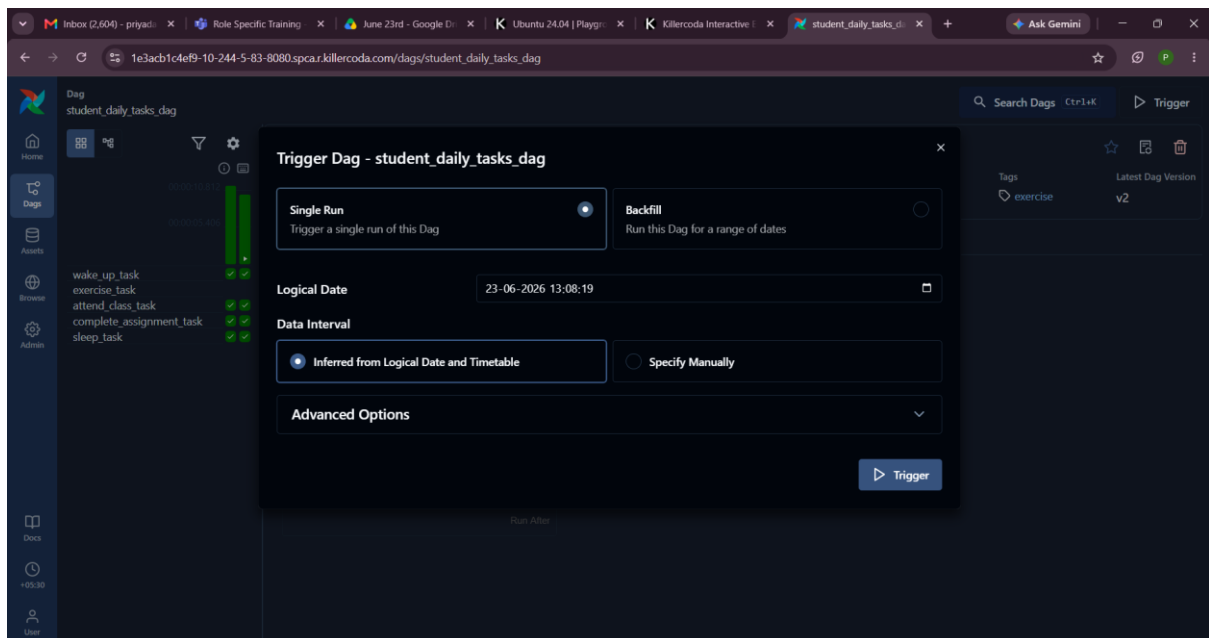
```

```
sleep_task = PythonOperator(
    task_id='sleep_task',
    python_callable=sleep
)
```

```
wake_up_task >> exercise_task >> attend_class_task >> complete_assignment_task >> sleep_task
```



Click trigger to execute the updated dag,



The screenshot displays the Apache Airflow web interface. The top navigation bar includes links for Home, Dags, Assets, Browse, and Admin. The main content area shows the 'student_daily_tasks_dag' DAG, which is in a 'Success' state. The 'Task Instances' tab is selected, displaying a table of task instances. The table columns are Task ID, Map Index, State, Start Date, Try Number, Operator, and Duration. The tasks listed are 'sleep_task', 'complete_assignment_task', 'attend_class_task', 'exercise_task', and 'wake_up_task', all of which are in a 'Success' state. The interface also includes a sidebar with navigation options and a top bar with the URL and browser tabs.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
sleep_task		Success	2026-06-23 13:08:38	1	PythonOperator	00:00:00.198
complete_assignment_task		Success	2026-06-23 13:08:37	1	PythonOperator	00:00:00.178
attend_class_task		Success	2026-06-23 13:08:36	1	PythonOperator	00:00:00.224
exercise_task		Success	2026-06-23 13:08:35	1	PythonOperator	00:00:00.159
wake_up_task		Success	2026-06-23 13:08:34	1	PythonOperator	00:00:00.275